

# Classes and Objects in Python

Track: Data Science | Level 1 - Day 12 | Tools: Python 3, Jupyter Notebook

## Level 1 - Day 12: Classes and Objects in Python

Track: Data Science

Task: Task 12 - Classes and Objects for Data Science

**Objective:** Understand Object-Oriented Programming (OOP) fundamentals in Python: defining classes, instantiating objects, instance variables, self, \_\_init\_\_ constructor, and class methods.

### 1. Quick Reference & Core Concepts

| OOP Concept      | Syntax / Keyword         | Data Science Application                                 |
|------------------|--------------------------|----------------------------------------------------------|
| Class Definition | class Student:           | Blueprint for entities (models, records, transformers)   |
| Constructor      | def __init__(self, ...): | State initialization (weights, learning rate, schema)    |
| Instance Method  | def evaluate(self):      | Data transformation, scoring, and calculation operations |
| Self Keyword     | self.attribute           | Reference to the current object instance                 |
| Class Attributes | total_records = 0        | Cohort-level counters and global configurations          |

### 2. Practical Code Examples

In [code]

```
# Example 1: Basic Class Definition and Object Instantiation
class Student:
    def __init__(self, student_id, name, score):
        self.student_id = student_id
        self.name = name
        self.score = score

s1 = Student("ST101", "Aman", 88.5)
print(f"Ex 1 - Instantiated Student: {s1.name} (ID: {s1.student_id}, Score: {s1.score}%")
```

In [code]

```
# Example 2: Adding Instance Methods for Dynamic Computation
class StudentRecord:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def calculate_average(self):
        return round(sum(self.marks) / len(self.marks), 2)

    def get_status(self):
        avg = self.calculate_average()
        return "Passed" if avg >= 50 else "Failed"

rec = StudentRecord("Sneha", [85, 90, 78])
print(f"Ex 2 - {rec.name}'s Avg: {rec.calculate_average()}%, Result: {rec.get_status()}")
```

In [code]

```
# Example 3: Class Variables vs Instance Variables
class Employee:
    company_name = "TechCorp Solutions" # Class attribute

    def __init__(self, emp_id, name, base_salary):
        self.emp_id = emp_id # Instance attribute
        self.name = name
        self.base_salary = base_salary

e1 = Employee(101, "Aditi", 95000)
print(f"Ex 3 - {e1.name} works at {Employee.company_name} with salary ${e1.base_salary:,}")
```

In [code]

```
# Example 4: String Representation with __repr__ and __str__
class DatasetBatch:
    def __init__(self, batch_id, size):
        self.batch_id = batch_id
        self.size = size

    def __str__(self):
        return f"Batch #{self.batch_id} (Elements: {self.size})"

batch = DatasetBatch(4, 128)
print(f"Ex 4 - Object String Display: {batch}")
```

### 3. Applied Data Science Pipeline

---

#### Applied Data Pipeline: Custom DataScaler Class Implementation

In [code]

```
class SimpleMinMaxScaler:
    def __init__(self):
        self.min_val = None
        self.max_val = None

    def fit(self, data):
        self.min_val = min(data)
        self.max_val = max(data)

    def transform(self, data):
        return [(x - self.min_val) / (self.max_val - self.min_val) for x in data]

raw_scores = [45, 60, 85, 95, 30]
scaler = SimpleMinMaxScaler()
scaler.fit(raw_scores)
scaled_scores = scaler.transform(raw_scores)
print(f"Raw Marks: {raw_scores}")
print(f"Scaled Marks [0, 1]: {[round(x, 3) for x in scaled_scores]}")
```